



SDU International Summer School

Introduction to R

Day 7 - Descriptive Statistics

Paola Vazquez-Castillo

11 August 2026

Contents

1	Introduction	2
2	Today's data	2
3	Summary statistics	2
3.1	Measures of location	3
3.1.1	Mean	3
3.1.2	Median	3
3.1.3	Mode	4
3.1.4	How the measures of location look in the data?	4
3.2	Measures of Spread	5
3.2.1	Range	5
3.2.2	Variance	6
3.2.3	Standard deviation	6
4	Describing a variable in one line: summary() and unique()	7
5	Reading a boxplot through its statistics	7
6	Tables	9
6.1	Two way tables	10
7	Measures of dependence (multiple variables together)	10
7.1	Covariance	10
7.2	Correlation coefficient	11
7.2.1	Theoretical example	11
7.3	Practical example: relationship between copies sold and revenue	14
8	From correlation to regression	15

9 Glossary	17
10 Exercise	17

1 Introduction

Statistics is the science that applies mathematics to convert data into useful information. In general, there are two approaches to this: 1) descriptive and 2) inferential.

Descriptive statistics are those tools that allow us to summarize (or describe) our data. That means, we can understand a group of data much faster and more easily than just staring at each row. Inferential statistics allow us to make predictions (inferences) from that data.

2 Today's data

The data `games.csv` reports the 2024 sales of videogames in steam for the 1500 most sold games. It includes the name of the game, the publisher class, the original retail price of the game, the rating given to the game based on critics, the average duration that players spent playing the game, the number of copies sold and the revenue.

```
library(tidyverse)
```

```
games <- read.csv("games.csv")
head(games)
```

```
##              name publisher_class price review_score
## 1          WWE 2K24             AAA  99.99           71
## 2    EARTH DEFENSE FORCE 6        Indie  59.99           57
## 3    Sins of a Solar Empire II    Indie  49.99           88
## 4      Legend of Mortal           Indie  19.99           76
## 5 Shin Megami Tensei V: Vengeance    AA  59.99           96
## 6          Soulmask             Indie  29.99           79
##   avg_playtime copies_sold revenue
## 1    42.36514    165301 8055097
## 2    29.65106    159806 7882151
## 3    12.45259    214192 7815247
## 4    24.79782    440998 7756399
## 5    34.25850    141306 7629252
## 6    95.69781    304685 7540563
```

3 Summary statistics

Summary statistics summarize a set of observations in order to communicate information in a simple way. These statistics can be divided into:

- Measures of location (or central tendency), such as the mean;
- Measures of spread (or statistical dispersion), such as the variance;
- Measures of the shape, such as skewness or kurtosis (not seen in this class);
- Measures of dependence between two or more variable, such as the correlation coefficient.

We want to describe the revenue and copies sold of the 1500 games, and later analyze the relationship between these two variables.

3.1 Measures of location

3.1.1 Mean

The mean is the average of the given data. It is calculated as the sum of all values divided by the number of observations. R has in built-in function to find the mean `mean()`.

```
### Calculating the mean manually ###  
revenue <- games$revenue  
sum(revenue)/length(revenue)
```

```
## [1] 2634106
```

```
mean(revenue)
```

```
## [1] 2634106
```

R will not skip missing values unless explicitly requested to do so. The mean of a vector with an unknown value is unknown (NA). However, you can give the `na.rm` argument (not available, remove) to request that missing values be removed.

There is only one slightly annoying exception: the `length` function will not understand `na.rm`, so we have to filter the values of the vector, identify the missing value with `is.na` and exclude them.

```
### Calculation the mean manually ###  
mean(revenue, na.rm = T)
```

```
## [1] 2634106
```

```
mean(games$copies_sold, na.rm = T)
```

```
## [1] 141574.9
```

3.1.2 Median

The median is the value that occurs exactly in the middle, once the values are ordered. R has also in build-in function to find the median `median()`.

There is slight hitch here because if the vector contains an even number of numbers, then there is no middle value. The logic here is that we need to work out the arithmetic average of the two values on either side of the middle.

```
median(revenue, na.rm = T )
```

```
## [1] 109304
```

```
#Copies sold  
median(games$copies_sold, na.rm = T )
```

```
## [1] 11972
```

Notice that the mean revenue is far larger than the median revenue. When the mean sits well above the median, it tells us the data are skewed to the right: a few very high values (here, a handful of blockbuster games) pull the mean up, while the median stays near the typical game.

3.1.3 Mode

The mode is the value that has the highest number of occurrences in the data set. R does not have a build-in function in R. We have to write our own function!

Step-by-step in simple terms:

1. **Find unique values**
Extract from the vector the unique values by removing duplicates.
2. **Count how often each value appears**
Check how many times each unique value shows up in the original list.
3. **Find the most frequent value**
Looks for the biggest appearance - that is, which value appears the most.
4. **Return the mode**
Gives back the most common value as the result - that's the mode.

```
# Create function to get the mode
mode_function<-function(values){

  #Find all unique values in the vector
  unique_values<-unique(values)

  #Find the number of times each unique values is present
#The match function returns a vector of positions of (first) matches
#of its first argument in its second.
#Tabulate counts the frequency of occurrence
  tab<-tabulate(match(values, unique_values))

  #Find the unique values with the most occurrence in the vector tab
  mode<-unique_values[tab == max(tab)]

  #Return the mode
  return(mode)
}

# Most common price

mode_function(games$price)
```

```
## [1] 19.99
```

There can be more than one mode.

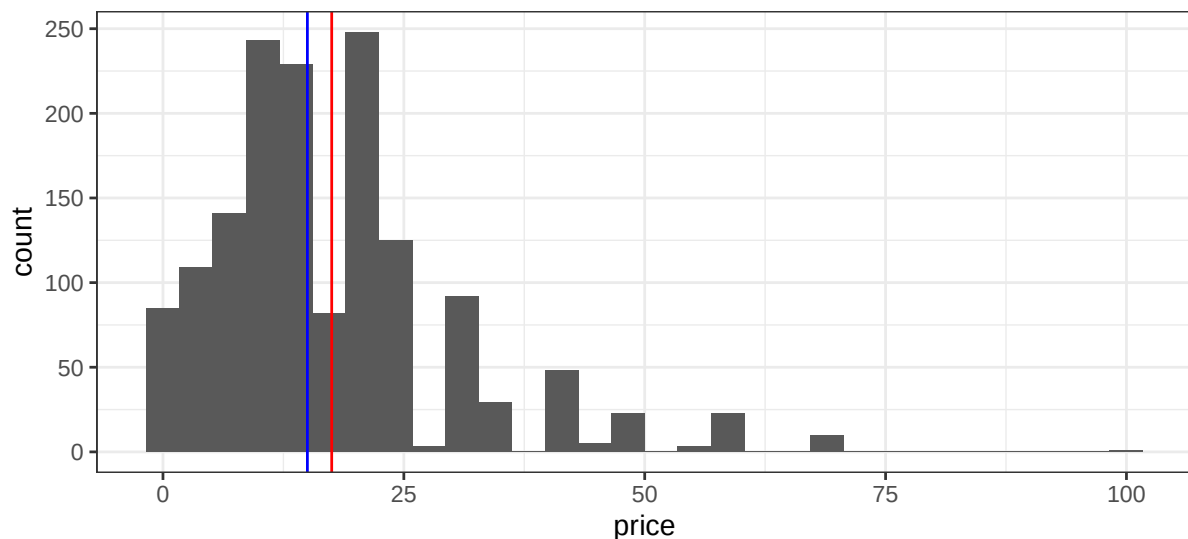
```
#Find the mode of the number of copies sold
mode_function(games$copies_sold)
```

```
## [1] 3127 11885 6151
```

3.1.4 How the measures of location look in the data?

```
games |>
  ggplot() +
  geom_histogram(aes(x=price))+
  geom_vline(aes(xintercept = mean(price)), col="red")+
  geom_vline(aes(xintercept = median(price)), col="blue")+
  theme_bw()
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value 'binwidth'.
```



3.2 Measures of Spread

3.2.1 Range

The range simply tells the range of values a variable can take, i.e. the minimum and maximum. These values can be found with the functions `min()`, `max()` and `range()`.

```
# Minimum
min(revenue, na.rm = T)
```

```
## [1] 20674
```

```
# Maximum
max(revenue, na.rm = T)
```

```
## [1] 837793356
```

```
# Range revenue
range(revenue, na.rm = T)
```

```
## [1] 20674 837793356
```

```
# Range price
range(games$price, na.rm = T)
```

```
## [1] 0.00 99.99
```

3.2.2 Variance

The variance measures how the data are spread out. It gives a rough idea of the spread. It is the average of the square differences from the mean. The mathematical function is:

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}$$

where x_i are the values, $\bar{x}$ is the mean and n is the number of observations.

The build-in function to calculate it is `var()`.

```
### Calculating the variance manually ###
```

```
#Find mean
```

```
Mean_revenue<-mean(revenue, na.rm = T)
```

```
#squared differences
```

```
sum((revenue-Mean_revenue)^2)/(length(revenue)-1)
```

```
## [1] 7.739213e+14
```

```
### Calculating the variance with build-in function ###
```

```
#Variance of revenue
```

```
var(revenue, na.rm = T)
```

```
## [1] 7.739213e+14
```

```
#Variance of price
```

```
var(games$price, na.rm = T)
```

```
## [1] 160.0063
```

3.2.3 Standard deviation

The standard deviation measures how spread are the data around the mean. It is the square root of the variance.

$$s = \sqrt{s^2}$$

The standard deviation describes the typical size of deviations from the mean and, unlike the variance, is expressed in the same units as the variable.

The build-in function is `sd()`.

```
### Calculating the standard deviation manually ###
```

```
sqrt(sum((revenue-Mean_revenue)^2)/(length(revenue)-1))
```

```
## [1] 27819440
```

```
### Calculating the standard deviation with build-in function ###
```

```
#SD of revenue
```

```
sd(revenue, na.rm = T)
```

```
## [1] 27819440
```

```
#SD of price
sd(games$price, na.rm = T)
```

```
## [1] 12.64936
```

4 Describing a variable in one line: summary() and unique()

summary() gives the location and spread of a numeric variable at once: the minimum, the three quartiles, the mean, the maximum, and the number of missing values.

```
summary(revenue)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
##  20674    45490   109304   2634106   455826 837793356
```

For a categorical variable, the quartiles make no sense. There unique() is the right tool, because it lists the distinct categories the variable takes.

```
unique(games$publisher_class)
```

```
## [1] "AAA" "Indie" "AA"
```

5 Reading a boxplot through its statistics

A boxplot starts from the five-number summary. Once you know the numbers, the picture reads itself.

The quartiles split the ordered data into four equal parts. The first quartile (Q1) is the value below which 25 percent of the data fall, the second (the median) sits at 50 percent, and the third (Q3) at 75 percent. The function quantile() returns them.

```
quantile(revenue, na.rm = TRUE)
```

```
##      0%      25%      50%      75%     100%
## 20674.0 45490.5 109304.0 455825.5 837793356.0
```

The interquartile range (IQR) is the distance between Q1 and Q3, so it is the spread of the middle half of the data. It is a measure of spread that ignores the extremes.

```
IQR(games$price, na.rm = TRUE)
```

```
## [1] 10
```

Together, the minimum, Q1, median, Q3, and maximum form the five-number summary, which is the same set of numbers summary() printed above. A boxplot draws them directly:

- the line inside the box is the median,
- the bottom and top of the box are Q1 and Q3, so the box height is the IQR,
- the whiskers reach out to the most extreme points that are still within 1.5 times the IQR of the box,
- any point beyond the whiskers is drawn separately and flagged as a possible outlier.

That last rule (a point is a possible outlier if it lies more than 1.5 times the IQR below Q1 or above Q3) is worth computing once by hand, so the dots on the plot are not a mystery.

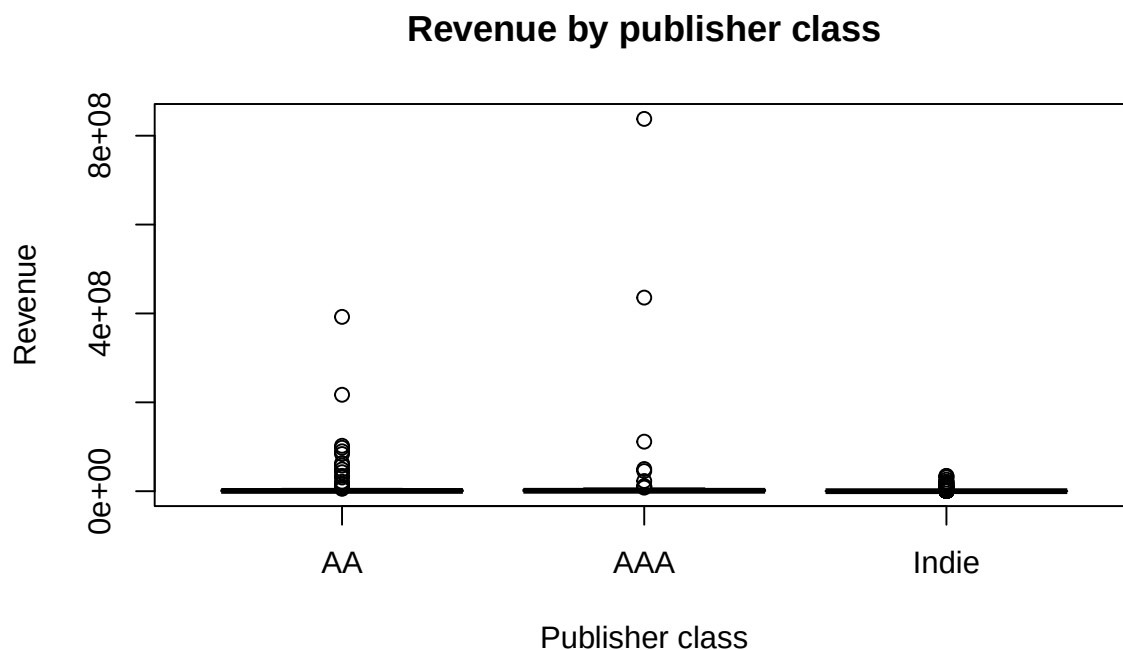
```
q1 <- quantile(revenue, 0.25, na.rm = TRUE)
q3 <- quantile(revenue, 0.75, na.rm = TRUE)
iqr <- q3 - q1
```

```
lower_fence <- q1 - 1.5 * iqr
upper_fence <- q3 + 1.5 * iqr
c(lower_fence, upper_fence)
```

```
##      25%      75%
## -570012 1071328
```

Now the plot.

```
boxplot(revenue ~ publisher_class, data = games,
        ylab = "Revenue",
        xlab = "Publisher class",
        main = "Revenue by publisher class")
```



Revenue is so skewed that a boxplot on the raw scale is unreadable: the box is squashed against the bottom and almost every blockbuster shows up as an outlier. This is itself a useful lesson, that heavily skewed data are hard to read, so we take the logarithm of revenue (the y axis).

```
#
boxplot(revenue ~ publisher_class, data = games,
        log= "y",
        ylab = "Revenue",
        xlab = "Publisher class",
        main = "Revenue by publisher class")
```




Reading this plot, you can see at a glance which type of publisher earns most (the highest median), which is most variable (the tallest box), and whether any group has unusually high or low earners (points beyond the whiskers).

6 Tables

Categorical data are usually described in the form of tables.

To count how many games fall in each category we use: `count()`

```
games |>
  count(publisher_class)
```

```
##   publisher_class    n
## 1             AA   146
## 2             AAA    52
## 3             Indie 1301
```

Turn counts into proportions by dividing by the total.

```
games |>
  count(publisher_class) |>
  mutate(proportion= n/sum(n))
```

```
##   publisher_class    n proportion
## 1             AA   146 0.09739827
## 2             AAA    52 0.03468979
## 3             Indie 1301 0.86791194
```

To describe a numeric variable within each group, combine `group_by()` with `summarise()`. This produces a small, tidy table of the same statistics we computed above, one row per group.

```
games |>
  group_by(publisher_class) |>
  summarise(n = n(),
            mean_rev = mean(revenue, na.rm = TRUE),
            median_rev = median(revenue, na.rm = TRUE),
            sd_rev = sd(revenue, na.rm = TRUE))
```

```
## # A tibble: 3 x 5
##   publisher_class      n mean_rev median_rev    sd_rev
##   <chr>          <int>    <dbl>    <dbl>    <dbl>
## 1 AA              146 10170533.    672187  40367198.
## 2 AAA              52 30511493.    928862 129953393.
## 3 Indie           1301   674120.    90795   2491655.
```

6.1 Two way tables

```
games <- games |>
  mutate(review_band = case_when(
    review_score >= 80 ~ "High",
    review_score >= 50 ~ "Medium",
    TRUE ~ "Low"
  ))
```

Two-way table of counts

```
games |>
  count(publisher_class, review_band) |>
  pivot_wider(names_from = review_band, values_from = n, values_fill = 0)
```

```
## # A tibble: 3 x 4
##   publisher_class High Low Medium
##   <chr>          <int> <int> <int>
## 1 AA              82   15    49
## 2 AAA             28    7    17
## 3 Indie           792  125   384
```

7 Measures of dependence (multiple variables together)

Measures of dependence indicate how closely two (or more) variables are related.

7.1 Covariance

The covariance measures the direction of a relationship between two variables (x and y). The mathematical formulas is

$$cov_{x,y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{n - 1}$$

The function to find the covariance is `cov()`.

7.2 Correlation coefficient

The coefficient of correlation measures the linear correlation between two variables. It measures how strong is the relation between the data.

The mathematical formula is

$$r = \frac{cov_{x,y}}{s_x s_y}$$

The correlation coefficient takes value between -1 and 1, where

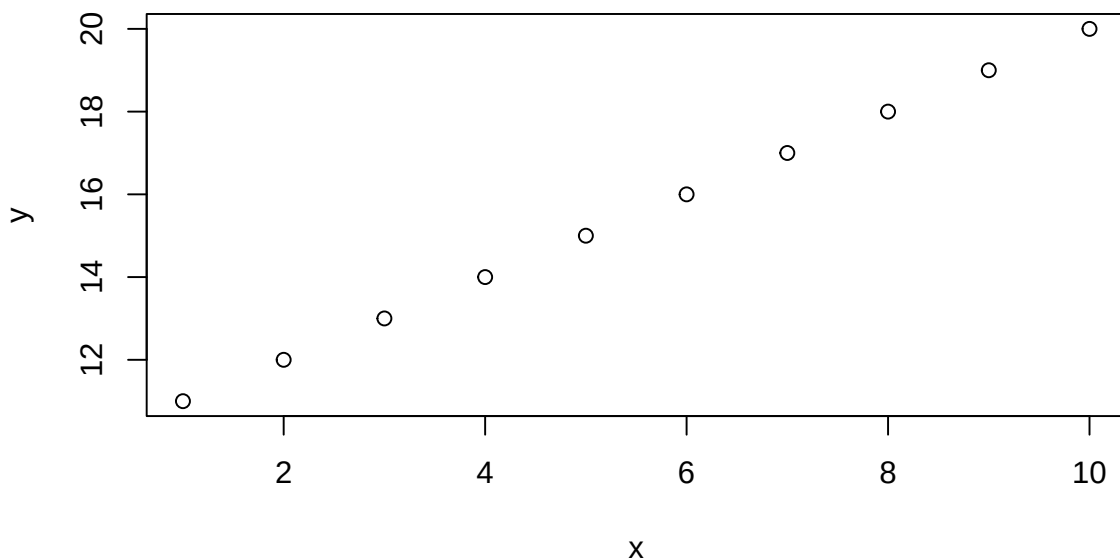
- 1 is a perfect positive relationship
- -1 is a perfect negative relationship
- 0 indicates no linear relationship

The function to find the correlation is `cor()`.

7.2.1 Theoretical example

```
### Example 1: Strong positive correlation
```

```
x<-c(1:10)
y<-c(11:20)
plot(x, y)
```



```
# Covariance
sum((x-mean(x))*(y-mean(y)))/(length(x)-1)
```

```
## [1] 9.166667
```

```
cov(x,y)
```

```
## [1] 9.166667
```

```
# Correlation
```

```
cov(x,y)/(sd(x)*sd(y))
```

```
## [1] 1
```

```
cor(x,y)
```

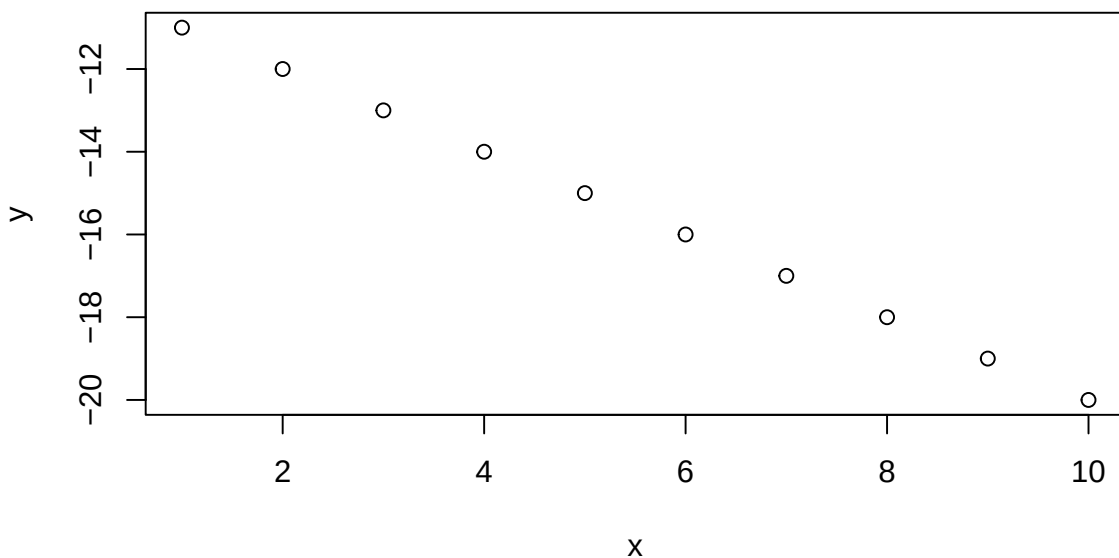
```
## [1] 1
```

```
### Example 2: Strong negative correlation
```

```
x<-c(1:10)
```

```
y<-c(-11:-20)
```

```
plot(x, y)
```



```
# Covariance
```

```
sum((x-mean(x))*(y-mean(y)))/(length(x)-1)
```

```
## [1] -9.166667
```

```
cov(x,y)
```

```
## [1] -9.166667
```

```
# Correlation
```

```
cov(x,y)/(sd(x)*sd(y))
```

```
## [1] -1
```

```
cor(x,y)
```

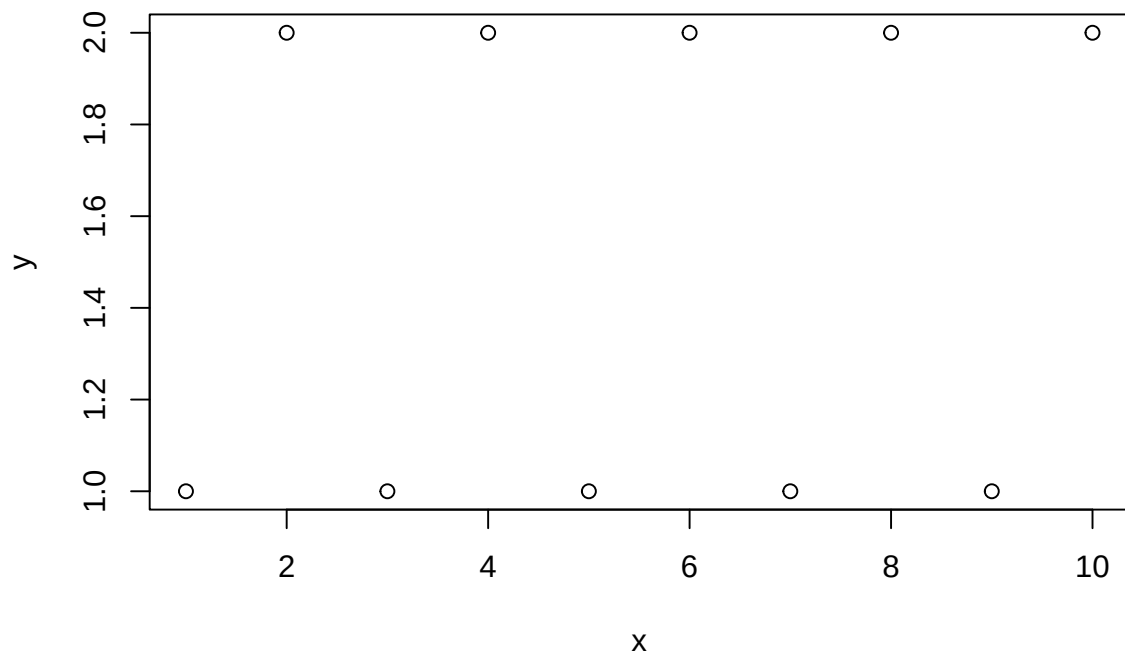
```
## [1] -1
```

```
### Example 3: Weak correlation
```

```
x<-c(1:10)
```

```
y<-rep(c(1,2), 5)
```

```
plot(x, y)
```



```
# Covariance
```

```
sum((x-mean(x))*(y-mean(y)))/(length(x)-1)
```

```
## [1] 0.2777778
```

```
cov(x,y)
```

```
## [1] 0.2777778
```

```
# Correlation
```

```
cov(x,y)/(sd(x)*sd(y))
```

```
## [1] 0.1740777
```

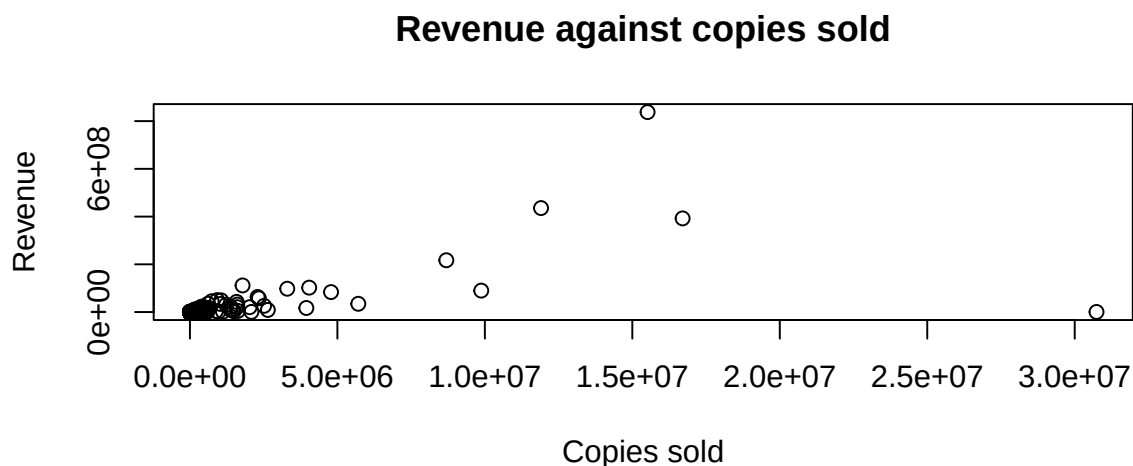
```
cor(x,y)
```

```
## [1] 0.1740777
```

7.3 Practical example: relationship between copies sold and revenue

Now we look at two real variables together. Studies of any market would expect that the more copies a game sells, the more revenue it earns. Always plot the two variables first, because the plot tells you the shape of the relationship, which decides how to read the number that follows.

```
plot(games$copies_sold, games$revenue,  
     xlab = "Copies sold",  
     ylab = "Revenue",  
     main = "Revenue against copies sold")
```



```
# Covariance  
cov(games$copies_sold, games$revenue)
```

```
## [1] 1.978693e+13
```

```
# Correlation  
cor(games$copies_sold, games$revenue)
```

```
## [1] 0.6276978
```

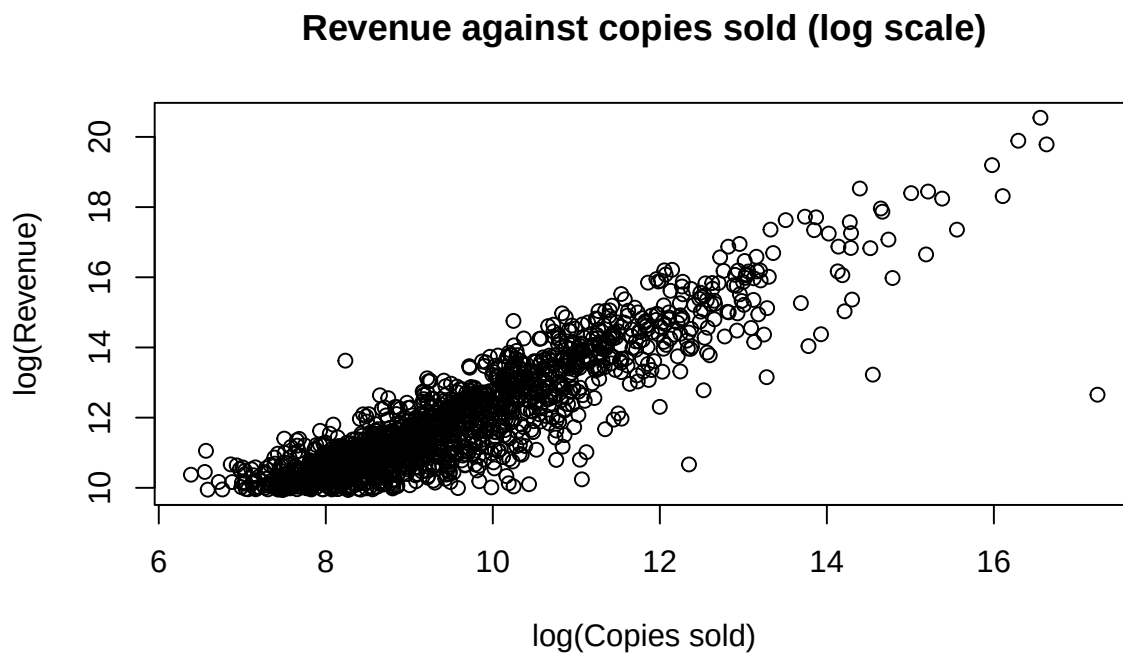
The correlation is positive, around 0.63, so more copies sold does go with more revenue. But the scatter plot is hard to read: almost every point is crushed into the bottom-left corner, while a few enormous games stretch both axes. The relationship is clearly there, yet the number feels lower than the picture suggests it should be. That is a sign that the scale, not the relationship, is the problem.

AI literacy. A correlation coefficient close to 0 does not always mean “no relationship”. It means no straight-line relationship. A strong curved or heavily skewed relationship can still produce a modest `cor()` value, with no error message, so the number looks trustworthy while hiding the pattern. Always plot the two variables before you trust the correlation. And remember that `cor()` returns `NA` without warning when the data contain missing values, unless you tell it what to do (for example, `use = "complete.obs"`).

Sometimes a relationship is hard to see because the data are very skewed. A few extreme values stretch the axes so much that the rest of the points are squashed together. Taking the logarithm

pulls in the long tail and spreads out the crowded values, so the relationship becomes clearer. Let us take the logarithm of both copies sold and revenue and plot again.

```
plot(log(games$copies_sold), log(games$revenue),  
     xlab = "log(Copies sold)", ylab = "log(Revenue)",  
     main = "Revenue against copies sold (log scale)")
```



```
# Correlation on the log scale  
cor(log(games$copies_sold), log(games$revenue))
```

```
## [1] 0.89651
```

The cloud is now a clear straight line, and the correlation has risen from about 0.63 to about 0.90. The relationship did not change, only the scale we measured it on. A word of caution, though. The transformation does not always help. It helps here because the raw data are badly skewed. When a relationship is already roughly straight, logging it will not raise the correlation, and may even lower it.

8 From correlation to regression

Correlation and simple linear regression are closely connected ways of describing a linear relationship. The correlation tells you how strong it is and in which direction. The regression line, which you will fit tomorrow, describes the line itself, so you can predict one variable from another.

First, the square of the correlation, r^2 , is the share of the variation in one variable that is linked to the other. It runs from 0 to 1 and is often read as a percentage.

```
r <- cor(log(games$copies_sold), log(games$revenue))  
r^2
```

```
## [1] 0.8037302
```

An r squared near 0.80 means that about 80 percent of the variation in (log) revenue lines up with (log) copies sold.

Second, the slope of the best-fitting straight line is built from the pieces we already have: the correlation, and the two standard deviations.

```
sd_y <- sd(log(games$revenue))
sd_x <- sd(log(games$copies_sold))

slope <- r * sd_y / sd_x
slope
```

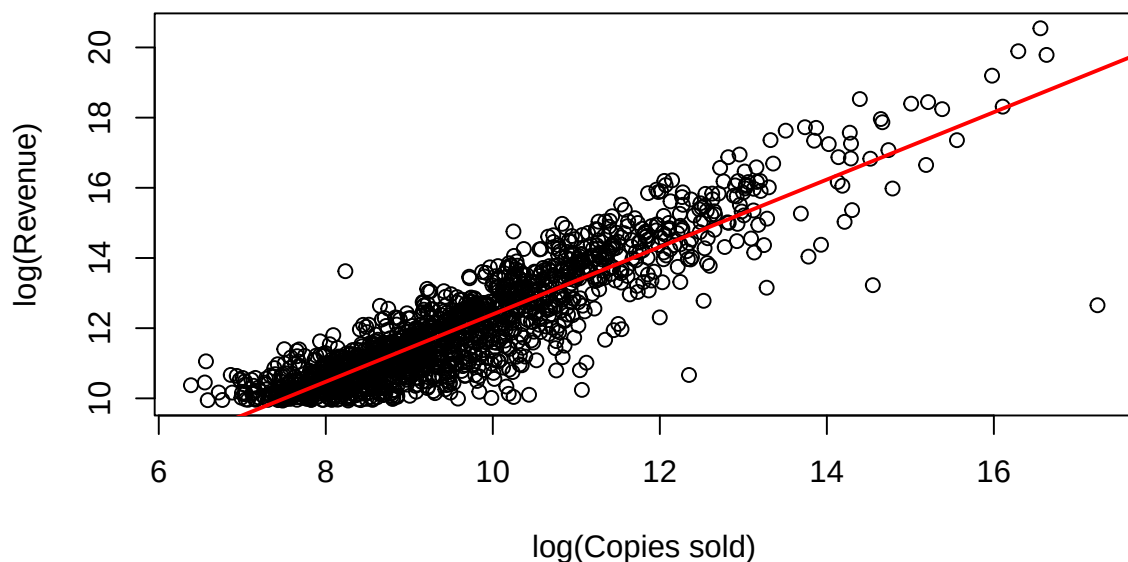
```
## [1] 0.9600718
```

To see where we are heading, here is the scatter plot on the log scale with that best-fitting line drawn on top. We use `lm()` here only to draw the line, and we unpack what it does tomorrow.

```
plot(log(games$copies_sold), log(games$revenue),
     xlab = "log(Copies sold)",
     ylab = "log(Revenue)",
     main = "The line we fit tomorrow")

abline(lm(log(revenue) ~ log(copies_sold), data = games),
       col = "red", lwd = 2)
```

The line we fit tomorrow



The straighter the relationship, the better this line fits, which is why transforming a skewed variable first can matter before fitting a regression.

9 Glossary

The functions used today, grouped by purpose.

#	Function	What it does
<i>Location</i>		
1	<code>mean()</code>	Average of a numeric vector
2	<code>median()</code>	Middle value of an ordered vector
<i>Spread</i>		
3	<code>min()</code> , <code>max()</code> , <code>range()</code>	Smallest, largest, and both together
4	<code>var()</code>	Variance (average squared distance from the mean)
5	<code>sd()</code>	Standard deviation (square root of the variance)
<i>Distribution and boxplots</i>		
6	<code>quantile()</code>	Quartiles and other percentiles
7	<code>IQR()</code>	Interquartile range (Q3 minus Q1)
8	<code>summary()</code>	Five-number summary plus mean, and count of NA
9	<code>unique()</code>	Distinct values of a variable
10	<code>boxplot()</code>	Draws the five-number summary, one box per group
<i>Describing groups</i>		
11	<code>count()</code>	Counts observations per category
12	<code>group_by()</code>	Declares groups for the next operation
13	<code>summarise()</code>	Collapses each group to summary statistics
<i>Relationship and regression</i>		
14	<code>cov()</code>	Covariance (direction of a relationship)
15	<code>cor()</code>	Correlation coefficient (strength and direction, -1 to 1)
16	<code>log()</code>	Logarithm, used to straighten a skewed relationship

Remember: functions that need missing values removed take `na.rm = TRUE`, and `cor()` takes `use = "complete.obs"`.

10 Exercise

Open the dataset `housing.csv`, which records sales of Danish homes in 2024. For each sale we know the type of home, its region, the year it was built, the number of rooms, its size in square metres (`sqm`) and its price. You have not seen this data before, so treat it as you would exam data.

- Describe the variable `price`. Find its mean, median, and standard deviation. Do the mean and median agree, and what does that tell you about the shape of the variable?
- Produce the summary of `price`. Then draw a boxplot of `price` split by `region`. Which region is most expensive, which is most variable, and are there any outliers?
- Build a grouped summary table giving the number of homes, the mean, and the standard deviation of `price` for each type of home.

- Studies would expect that larger homes cost more. Plot the relationship between `sqm` and `price`, and find their correlation coefficient. What is the direction and strength of the relationship?
- Try a log transformation of `price`, plot the relationship again, and recompute the correlation. Did it change? Explain why, or why not, and what this tells you about describing a relationship with a single correlation coefficient.